

# Managed vs. External Tables: Erstellen, Ändern, Löschen, Konvertieren

Section 7 · Governance and Security · Quelle: Databricks-Dokumentation (Unity Catalog Tables)

---

Ein früheres Kapitel dieser Section hat den konzeptionellen Unterschied zwischen Managed und External Tables bereits eingeführt. Für die Prüfung ist zusätzlich das praktische Handwerkszeug wichtig: die konkreten SQL-Befehle, um beide Tabellentypen anzulegen, zu ändern, zu löschen und ineinander umzuwandeln.

## 1. Managed Tables anlegen, ändern, löschen

Wird beim Anlegen einer Tabelle kein `LOCATION` angegeben, entsteht automatisch eine **Managed Table**: Unity Catalog verwaltet sowohl die Metadaten als auch den zugrunde liegenden Speicherort innerhalb des von Databricks verwalteten Cloud-Speichers.

```
-- Managed Table anlegen (kein LOCATION angegeben)
CREATE TABLE main.sales.orders (
  order_id BIGINT,
  customer_id BIGINT,
  order_date DATE,
  amount DOUBLE
);

-- Ändern: Spalte hinzufügen bzw. umbenennen
ALTER TABLE main.sales.orders ADD COLUMN currency STRING;
ALTER TABLE main.sales.orders RENAME COLUMN amount TO order_amount;

-- Löschen: entfernt Metadaten UND die zugrunde liegenden Daten
DROP TABLE main.sales.orders;
```

## 2. External Tables anlegen, ändern, löschen

Eine **External Table** entsteht, sobald beim Anlegen ein expliziter `LOCATION` -Pfad in einem extern registrierten Storage-Bereich (Unity-Catalog-*External Location*) angegeben wird. Unity Catalog

verwaltet dann nur die Metadaten – die Daten selbst bleiben unter der vollen Kontrolle des zugrunde liegenden Cloud-Speichers.

```
-- External Table anlegen (mit explizitem LOCATION)
CREATE TABLE main.sales.orders_ext (
  order_id BIGINT,
  customer_id BIGINT,
  order_date DATE,
  amount DOUBLE
)
LOCATION 's3://my-company-data/sales/orders_ext/';

-- Löschen: entfernt NUR die Metadaten, die Daten unter dem LOCATION-Pfad bleiben erhalten
DROP TABLE main.sales.orders_ext;
```

Genau dieser Unterschied beim `DROP TABLE`-Verhalten ist einer der am häufigsten geprüften Punkte: Bei Managed Tables verschwinden mit dem `DROP` auch die Daten unwiderruflich, bei External Tables bleiben sie bestehen und könnten – sofern noch bekannt – erneut über `CREATE TABLE ... LOCATION` eingebunden werden.

### 3. Zwischen Managed und External konvertieren

Seit neueren Databricks-Runtime-Versionen lässt sich eine bestehende External Table direkt in eine Managed Table umwandeln, ohne die Daten manuell kopieren zu müssen:

```
-- External Table nachträglich in eine Managed Table umwandeln
ALTER TABLE main.sales.orders_ext SET MANAGED;
```

Dabei bleiben Tabellename, Berechtigungen, abhängige Views und die vollständige Tabellenhistorie (Time Travel) erhalten – die Konvertierung ist zudem so konzipiert, dass sie parallel zu laufenden Lese-/Schreibzugriffen mit minimaler Unterbrechung durchgeführt werden kann. Eine Rückkonvertierung von Managed zu External ist bewusst nicht als einfacher Befehl vorgesehen, da damit die Verantwortung für die physische Datenverwaltung wieder aus Unity Catalog herausgegeben würde.

### 4. Entscheidungshilfe

| Kriterium | Managed Table | External Table |
|-----------|---------------|----------------|
|-----------|---------------|----------------|

|                                       |                                               |                                                                             |
|---------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------------|
| Wer verwaltet die Daten?              | Databricks/Unity Catalog                      | Bestehender Cloud-Speicher, außerhalb von Databricks kontrolliert           |
| Verhalten bei <code>DROP TABLE</code> | Daten werden mitgelöscht                      | Nur Metadaten werden entfernt                                               |
| Empfohlen für                         | Neue Tabellen (Standard- und Empfehlungsfall) | Bestehende Daten, die auch von Systemen außerhalb Databricks gelesen werden |
| Unterstützte Formate                  | Delta Lake, Apache Iceberg                    | Delta, CSV, JSON, Avro, Parquet, ORC, Text                                  |

**Ergänzt aus der offiziellen Databricks-Dokumentation:** Managed Tables gelten laut Databricks als Standard- und empfohlener Tabellentyp für Delta Lake und Apache Iceberg, da Unity Catalog dabei sämtliche Lese-, Schreib-, Speicher- und Optimierungsverantwortung übernimmt (inkl. automatischem `OPTIMIZE / VACUUM` über Predictive Optimization, siehe Section 6). Die Konvertierung per `ALTER TABLE ... SET MANAGED` unterstützt außerdem ein Rollback: Eine bereits konvertierte Managed Table kann bei Bedarf wieder in eine External Table zurückversetzt werden.  
Quelle: [Convert external or foreign Delta Lake tables to Unity Catalog managed tables – Databricks-Dokumentation](#)